```ocaml
(*
                            CS 51 Lab 19
                Reaction-Diffusion Cellular Automaton

    ***************************************************************
    WARNING: If you have photosensitive epilepsy, you should avoid
    viewing the reaction-diffusion cellular automaton. It exhibits
    rapidly flashing visual patterns.
    ***************************************************************
 *)

module G = Graphics ;;

(* Automaton parameters *)
let cGRID_SIZE = 100 ;;      (* width and height of grid in cells *)
let cSPARSITY = 5 ;;         (* inverse of proportion of cells initially live *)

(* Rendering parameters *)
let cCOLOR_LEGEND = G.rgb 173 106 108 ;;  (* color for textual legend *)
let cSIDE = 8 ;;             (* width and height of cells in pixels *)
let cRENDER_FREQUENCY = 1    (* how frequently grid is rendered (in ticks) *) ;;
let cFONT = Some "-adobe-times-bold-r-normal--34-240-100-100-p-177-iso8859-9"

type rd_state = float

(* offset index off -- Returns the `index` offset by `off` allowing
   for wraparound. *)
let rec offset (index : int) (off : int) : int =
  if index + off < 0 then
    cGRID_SIZE - (offset ~-index ~-off)
  else
    (index + off) mod cGRID_SIZE ;;

(* rd_update grid i j -- Returns the updated value for cell at `i, j`
   in the grid based on some simple reaction diffusion rules. *)
let rd_update (grid : rd_state array array) (i : int) (j : int) =
  let neighbors = ref 0. in
  for i' = ~-1 to ~+1 do
    for j' = ~-1 to ~+1 do
      let i_ind = offset i i' in
      let j_ind = offset j j' in
      neighbors := !neighbors +. grid.(i_ind).(j_ind)
    done
  done;
  let norm = (!neighbors -. grid.(i).(j)) /. 8. in
  let reacted = norm -. 4.
                        *. (norm -. 0.1)
                        *. (norm -. 0.5)
                        *. (norm -. 0.9) in
  let clipped = min 1. (max 0. reacted) in
  clipped ;;
```

```
(* rd_initialize grid -- Fills `grid` with uniform random values in [0, 1]. *)
let rd_initialize (grid : rd_state array array) : unit =
  for i = 0 to cGRID_SIZE - 1 do
    for j = 0 to cGRID_SIZE - 1 do
      grid.(i).(j) <- Random.float 1.
    done
  done

module RDSpec : (Cellular.AUT_SPEC
                 with type state = rd_state) =
  struct
    type state = rd_state
    let name : string = "Turing's RD"
    let initial : state = 0.1
    let grid_size : int = cGRID_SIZE
    let initialize = rd_initialize
    let update = rd_update
    let cell_color (cell : state) : G.color =
      let col = (int_of_float (cell *. 255.)) in
      if not ( 0 <= col && col < 256 ) then
        (Printf.printf "%f -> %d\n" cell col;
         raise (Invalid_argument "color out of range"))
      else
        G.rgb col col col
    let side_size : int = cSIDE
    let legend_color : G.color = cCOLOR_LEGEND
    let font : string option = cFONT
    let render_frequency : int = cRENDER_FREQUENCY
  end ;;

module Aut = Cellular.Automaton (RDSpec) ;;

(*......................................................................
  Running the automaton and displaying the results
 *)

(* main seed -- Runs the automaton using the provided random `seed`
   (for replicability), displaying updates to the graphics window. *)
let main seed =

  Random.init seed;      (* initialize randomness *)
  Aut.graphics_init ();  (* ... and graphics window *)

  Aut.run_grid (Aut.initialized_grid ()) ;;

(* Run the automaton with user-supplied seed *)
let _ =
  if Array.length Sys.argv <= 1 then
    Printf.printf "Usage: %s <seed>\n  where <seed> is integer seed\n"
                  Sys.argv.(0)
  else
    main (int_of_string Sys.argv.(1)) ;;
```